

融合算子优化

fused_add_rmsnorm_group_quant

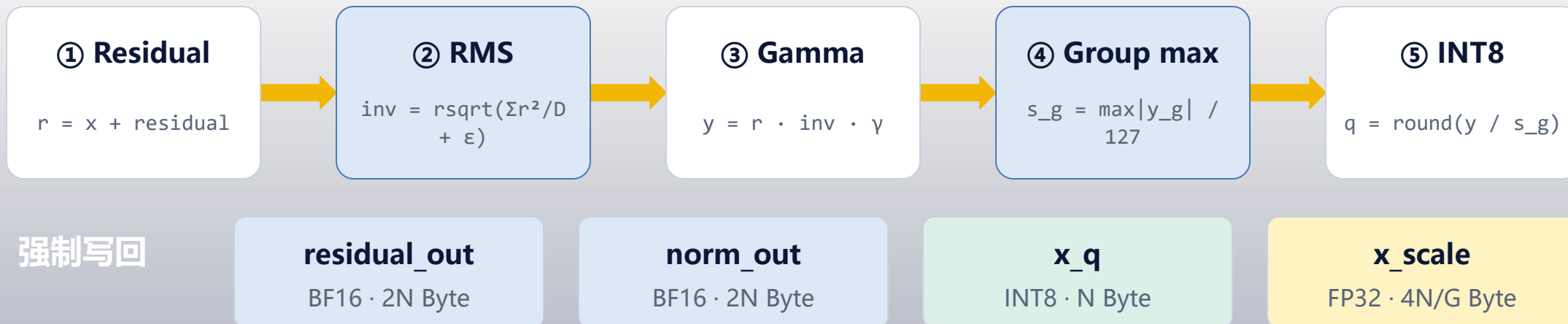
姓名：王鑫培

职务：天津大学 学生

题目拆解：四路强制输出决定优化下界

FlagOS

同一行完成 residual、RMSNorm、分组量化；额外全量读写直接增加 HBM 流量



10 组 Workload | M = 1–8192 | D = 4096–8192 | group_size = 128 / 256 | 6 种后端

总体方案：统一数学，六芯分路

FlagOS

后端决定所有权， $M \times D$ 决定执行结构；参数服从资源约束



固定层：所有后端完全一致

数学公式 | FP32 归约 | 四路输出 | 精度约束

专用层：按架构与 Workload 改变

所有权 | segment | row tile | warps | stages

先用理论排除错误方向，再把搜索预算集中到执行结构。

瓶颈分析: AI=0.665, 确定带宽瓶颈

FlagOS

AI $\ll$ ridge point: 优化对象是字节流量, 而不是少量算术指令

理想全局内存字节账本

输入: $x \cdot 2N + \text{residual } 2N + sy \cdot D$

输出: $2N + 2N + N + 4N/G$

$$B_{\min} = 9N + 4N/G + sy \cdot D$$

$$F \approx 6N + 3M + 2N/G$$

$$AI = F / B_{\min} \approx 0.665 \text{ FLOP/Byte}$$

A100 FP32 ridge point

$$19.5 \text{ TFLOP/s} \div 1.555 \text{ TB/s} = 12.54 \text{ FLOP/Byte}$$

本算子

 0.665

ridge

 12.54

AI $\ll$ ridge
 $\Rightarrow$ 大 Workload 受 HBM 带宽限制

禁止第二遍 | 不重读 x / residual

禁止大 scratch | 不写 N 级中间张量

提高驻留行 | 缩短 live range

计算口径: $N=M \times D$, $G=\text{group_size}$, $sy=2 \text{ Byte}$; A100 FP32 19.5 TFLOP/s, HBM 1.555 TB/s

芯片差异：相同数学必须映射为不同执行图

warp/wave、CU/SM 数量与后端 lowering 决定所有权、分段和调度参数

平台	确认型号	关键执行特征	最终代码中的所有权
Ascend	Ascend 910B4	双 AIV; 约 61 GiB	sid 拆奇偶行; grid=M/2
NVIDIA	A100-SXM4-40GB	warp 32; 108 SM	ROW_TILE=2; W8
MetaX	MetaX C550	wave 64; 104 CU	W4; 长 D 用 SEG=4096
T-Head	PPU-ZW810E	warp 32; 64 CU	W8; direct INT8
TianShu	Iluvatar BI-V150	wave 64; 16 CU	分层 max; W8
Hygon	BW / gfx936	80 CU; HIP 6.1	SEG=2048; W4

统一：公式与精度 | 专用：row ownership、wave reduction、segment、warps、stages

优化一：单遍 Streaming, 读一次写四路

FlagOS

r 同时进入平方和、gamma 加权、group max; 归约后直接写四路输出

① 单遍加载

→ $r = x + \text{residual}$
 $w = r \cdot \gamma$

② 片上状态

→ $\text{sum}(r^2)$
 $\text{group_max}(|w|)$

③ 一次归一化

→ $\text{inv} = \text{rsqrt}(\text{sum}/D + \epsilon)$
 $\text{scale} = \text{max} \cdot \text{inv} / 127$

④ 直接写回

$\text{residual_out} / \text{norm_out}$
 $\text{x_q} / \text{x_scale}$

1x

x / residual 全量读取次数

```
r = load(x) + load(residual)
w = r * load(gamma)
ss = sum(r * r)
gmax = group_max(abs(w), G)
inv = rsqrt(ss / D + eps)
scale = max(gmax * inv / 127, 1e-12)
norm = w * inv
store(residual_out, r)
store(norm_out, norm)
store(x_scale, scale)
store(x_q, round(norm / scale))
```

0

N 级 GM scratch

实现: preweighted full-row / segmented streaming kernels

优化二：Segment 与 Row Tile 的资源权衡

FlagOS

SEGMENT 降低向量宽度；ROW_TILE=2 以双行状态换 gamma 复用

Full-row

小 M / 特殊回退

BLOCK_D = next_pow2(D)

- 一个 program 负责一行
- 单次宽向量完成归约
- 简单；可能包含 padding lanes

Segmented streaming

大 M / 大 D 主路径

lane domain = SEGMENT

- SEGMENT = 2048 / 4096
- square_acc 跨段折叠后归约一次
- 缩小 lowering 与 masked-lane 域

ROW_TILE = 2

NVIDIA 特定 Workload

state $\approx$ 2 rows

- 一个 program 处理两行
- gamma 每段加载一次并广播
- 仅 $M \geq 512$ 且 $D \leq 6144$

收益条件：gamma/launch 节省 > 双行状态带来的寄存器与 occupancy 代价

最终代码：SEGMENT=2048/4096; NVIDIA ROW_TILE=2

优化三：Ascend 双 AIV 分行, Grid 减半

FlagOS

sub_vec_id() 把同一 owner 映射到两个 AIV; 每个 AIV 独立完成一行

```
owner = tl.program_id(0)
sid = tle.dsa.ascend.sub_vec_id()
row = owner * 2 + sid
```

```
grid = (cdiv(M, 2),)
kernel[grid](...,
  SEGMENT=4096,
  num_warps=8,
  num_stages=1,
  multibuffer=False)
```

一个 program owner → 两个物理 AIV

AIV 0 • sid=0

row = 2 • owner

AIV 1 • sid=1

row = 2 • owner + 1

各自完成 residual → RMS → norm → quant

Grid

M → ceil(M/2)

通信

0 次跨 AIV / 0
scratch

官方最佳

Ascend 5.98×

优化四：让后端、M、D 共同决定执行路径

FlagOS

Dispatcher：专用路径优先；其他 shape 进入精确 stable fallback

后端 / 条件	执行路径	关键参数	解决的资源约束
Ascend • M≥8	双 AIV 行所有权	SEG=4096 · W8 · S1	同时占用两 AIV；grid 减半
NVIDIA • M≥512 D≤6144	parallel row-group	ROW_TILE=2 · SEG=2048	gamma 复用 > 寄存器增量
Hygon • M≥128	segmented row	SEG=2048 · W4	固定 2048-lane 域；改善 lowering
TianShu • M≥512	wave64 hierarchical max	W8 · stages=1/2	匹配 wave64 与 16 CU
T-Head • M≥128 排除 128×4096	segmented streaming	SEG=2048 · W8 · INT8	直接量化写回；避免中间 cast
MetaX • D=7168/8192	stable segmented	SEG=4096 · W4	减少分段/helper 次数
其他 / 小 M	stable full-row	next_pow2(D)	避免分段启动开销；保证精确回退

优先级：硬件专用路径 → 大 Workload 结构路径 → 精确 stable fallback

消除 wave mismatch、冗余转换与不匹配的流水深度

TianShu • wave64 max

$G=128/256 \rightarrow 2/4$ waves

先 wave 内 max, 再合并 wave maxima

T-Head • direct INT8

$q = \text{round}(\text{norm} / \text{scale})$

直接写 INT8; 不保留中间浮点量化张量

MetaX • long-D segment

$D \in \{7168, 8192\} \rightarrow \text{SEG}=4096$

分段数下降; 减少 helper 与边界处理

W4 / W8 分离

Hygon, MetaX $\rightarrow$ W4; 其余 $\rightarrow$ W8

在并行归约与寄存器占用之间取平衡

stage 按 D 选择

TianShu: $D=4096/7168 \rightarrow S1$; else S2

流水深度服从资源占用, 不盲目增大

精确 stable fallback

small M / unmatched shape

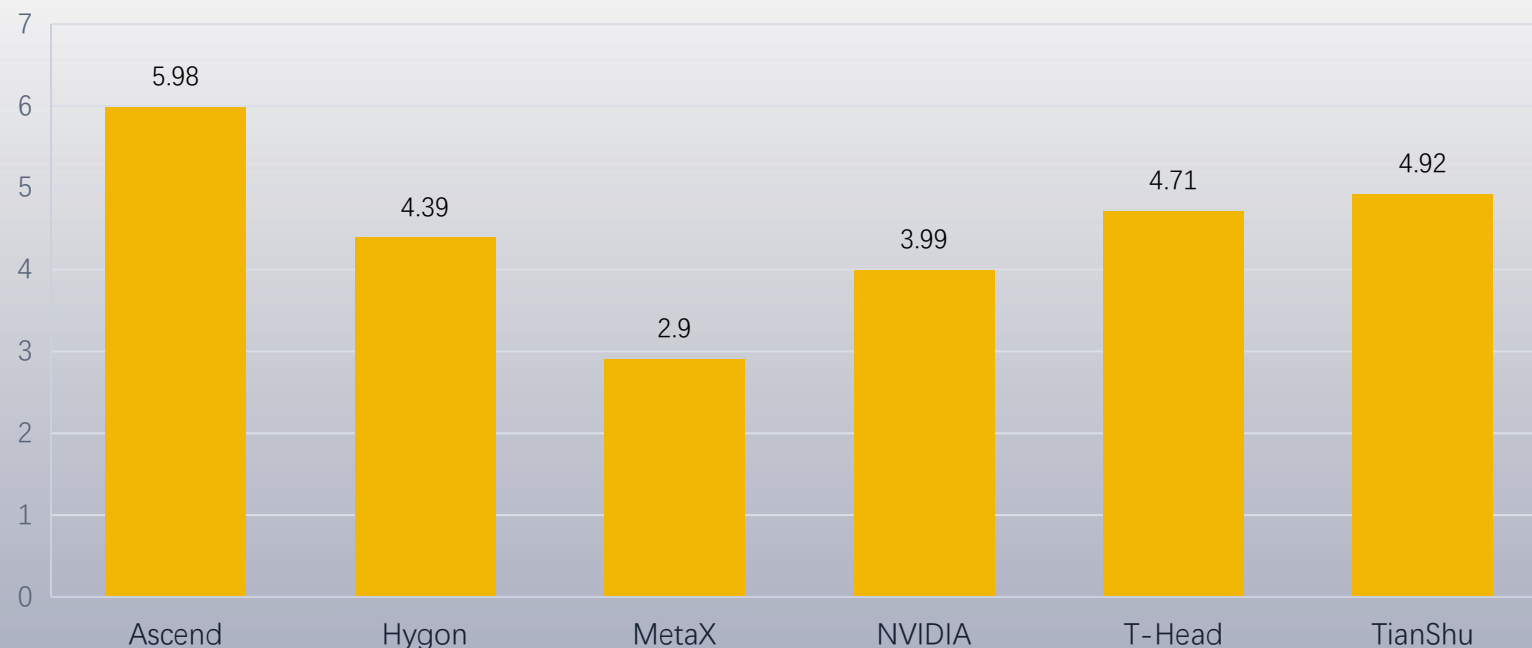
保留低启动开销路径; 隔离专用优化风险

原则：只有能解释资源变化、且通过分平台官方评测的参数，才进入综合版。

官方结果：六平台通过，Task 01 第一

FlagOS

官方最终成绩：六平台 6/6 通过；六芯片几何平均加速比 4.38×



4.38×

六芯片几何平均

#1

Task 01 最终排名

6 / 6 全平台通过

来源：FlagOS KernelGen 上海站最终官方榜单

把时间花在减少未知：建证据、做诊断、攻结构、及时合并综合版

6/6

六平台全部通过

#1

Task 01 最终排名

4.38×

六芯片几何平均

72 小时时间分配



截止前最后 20%：停止大分支 → 合并收益 → 官方验证 → 保存证据

先建证据	题面 / SHA / 分项结果
再做诊断	字节账本 / Roofline
优先结构	所有权 / 访存次数
持续归因	单变量 / 分平台

踩坑经历

- 调参先于瓶颈诊断 BLOCK / warps 反复试错、边际收益低 → 先做字节账本，优先改访存与核结构
- 截止前仍分散探索 单点收益未及时进入综合版 → 滚动合并；最后 20% 只验证与归档

获奖代码已开源：<https://github.com/sinpeyw/flagos-kernelgen-72h-shanghai-2026>

谢 谢

获奖代码已开源: <https://github.com/sinpeyw/flagos-kernelgen-72h-shanghai-2026>



公众号



FlagOS仓地址

众智FlagOS: 面向多元 AI 芯片的开源智算软件系统

- 高性能通用AI 算子库 FlagGems <https://github.com/flagos-ai/FlagGems>
- 统一 AI 编译器 FlagTree: <https://github.com/flagos-ai/flagtree>
- 并行训推一体框架 FlagScale: <https://github.com/flagos-ai/FlagScale>
- 统一通信库 FlagCX: <https://github.com/flagos-ai/FlagCX>